

CSE340: Computer Architecture

Assignment 2

Question - 1:

Given a machine code, what do we understand by looking at the **Opcode** field?

Answer: The Opcode field helps figure out the instruction format/type. In some cases we can also identify the exact instruction by just looking at the opcode field.

Question - 2:

Suppose you have converted LD X9, 10(X21) to its equivalent RISC-V machine code. Now based on which **field** machine will understand the size of the data being loaded into register X9?

Answer: funct3

Question - 3:

Sd x15, 10(x16); Here x15 is considered as rs2 and x16 as rs1.

Why not the opposite explain your logic.

Hint: Remember the design decision?

Answer:

When it comes to converting an instruction to its machine code, developers had two options.

Size of all instructions will be same (32 bits)

Vs

All the instructions will follow the same format.

Although they chose the first one, yet they tried to maintain the second wherever possible.

In sd x15, 10(x16); x16 is treated as rs1 and x15 as rs2 because of the design choice in RISC-V to keep instruction formats as consistent as possible across different instruction types.

For example, in a load instruction:

ld rd, imm(rs1)

Here, rs1 is the base address register.

To keep the store instruction similar in structure, RISC-V uses:

sd rs2, imm(rs1)

So in store instructions:

rs1 = base address register

rs2 = data register whose value will be stored

Thus, in sd x15, 10(x16); x16 is rs1 because it holds the memory base address

x15 is rs2 because its value is being written to memory; if they made it the opposite, the format would become less consistent with the load instruction.

Question - 4:

$$B[a] = [(A[B[3]] + B[24] - 12) * 28] / 256$$

a) Construct the equivalent RISC-V code of the above mentioned C code.
Base addresses of array A and B are in register X20 and X21; Consider both arrays as double words and variable a is mapped to register X22

Answer:

Question - 4.rvx ; Run this on the simulator.

b) From your converted codes, pick any instruction from each of the three formats(R, I, S) and convert them into their corresponding binary values.

Note: For opcode, funct3 or any other field values that you do not know just put "X"

Make sure you write the code in such a way that there is atleast one R or I or S type instruction.

Answer:

Once you run the code on the simulator, on the Log panel you will be able to see all the instructions and their machine codes.

Question - 5:

What is the significance of the funct3 field in case of a Load instruction?

Answer: Machine will understand the size and signedness of the data being loaded.

Question - 6:

Suppose we are creating a custom instruction.

`fil x5, 10, x8;`

This instruction 'fil' copies the least significant 10 bits of register x5 into register x8.

Now based on this information, which instruction format from risc v will be a best fit for this instruction?

Answer:

`x5 = rs` ; as we are copying data from here

`x8 = rd`; as we are writing data here

`10` = is an immediate or constant

So we have a register source, a register destination and an immediate - which matches with the I type instruction. - Therefore, I type format will be the best fit for this instruction.

Question - 7:

Explain why the immediate or constant part is divided into two portions in S type instruction?

Answer: To somewhat match the instruction format with other instruction types.

	f7	rs2	rs1	f3	rd	Opcode
R type	f7	rs2	rs1	f3	rd	Opcode
I type	Imm		rs1	f3	rd	Opcode
S type		rs2	rs1	f3		Opcode

See, we have rs1 and rs2 in s type instruction, if we try to match the format with others then we don't have enough space to keep the 12 bit immediate intact altogether. Therefore, we divide it into two parts imm[11:5] and imm[4:0]
 Imm[11:5] sits in the position of f7 and
 Imm[4:0] sits in the position of rd
 Therefore, the position of rs1 and rs2 stays the same across different instructions.

Question - 8:

SB type instruction has two register sources and a 12 bit signed constant.
 S type instruction has two register sources and a 12 bit signed constant.
 If all the contents are the same, then why are categorized in two different formats.

Answer:

S type - sd x5, 10(x12)

SB type - beq x5, x6, label

Although, both have rs1 and rs2 and immediate is also a 12 bit signed constant but

- the way we represent the immediate is different.
- their purpose is different
- their encoding is different
- Also, For S type, we directly mention the immediate in the instruction but for SB type, we mention the label and the assembler calculates the immediate/offset itself.

Question - 9:

UJ type instruction has one register destination and a 20 bit signed constant.

U type instruction has one register destination and a 20 bit signed constant.

If all the contents are the same, then why are categorized in two different formats.

Answer:

U type - lui x5, 23

UJ type - jal, x1, label

Although, both have rd and immediate is also a 20 bit signed constant but

- the way we represent the immediate is different.
- their purpose is different
- their encoding is different
- Also, For U type, we directly mention the immediate in the instruction but for UJ type, we mention the label and the assembler calculates the immediate/offset itself.

Question - 10:

Assume addresses 0(x10) and 4(x10) contain $(10)_{10}$ and $(2)_{10}$, respectively. Register x10 contains $(12)_{10}$. Using this information, trace the RISC-V code below and show the register values after each instruction.

LW x5, 0(x10)	x5 = 10	x10 = 12
LW x6, 4(x10)	x6 = 2	x10 = 12
BGE x5, x6, L1	x5 = 10	x6 = 2
ADDI x5, x5, 2	x5 = N.E	
SW x5, 0(x10)	x5 = N.E	x10 = N.E
Beq x0, x0, Exit	x0 = N.E	
L1:		
ADDI x6, x6, -3	x6 = -1	
SW x6, 4(x10)	x6 = -1	x10 = 12
Exit:		

Note:

For any instruction that is not executed, write “N.E” in place of the register values.

Question - 11:

A RISC-V processor is executing the following instruction:

Address	Instruction	Machine Code
0x00001020	beq X ₅ , X ₆ , label	0x8062A463

Now, answer the following questions:

- i. **Extract** the immediate field from the instruction and **reconstruct** the signed branch offset. Your final answer should be a signed **decimal** number.

Answer:

0x8062A463 = 1000 0000 0110 0010 1010 0100 0110 0011

1	000 000	00110	00101	010	0100	0	110 0011
imm[12]	imm[10:5]	rs2	rs1	f3	imm[4:1]	imm[11]	Opcode

	imm[12]	imm[11]	imm[10:5]	imm[4:1]
Immediate	1	0	000 000	0100

Immediate = 1000 0000 0100

Offset = Immediate x 2

Here the msb bit of the immediate is 1, meaning it is a negative number. So, performing 2s com. on the immediate to find its value.

$0111\ 1111\ 1011 + 1 = 0111\ 1111\ 1100 = (2044)_{10}$

So the immediate = -2044

Offset = -2044 x 2 = -4088

- ii. Calculate the target address of the branch assuming the branch condition is satisfied. Your final answer should be a decimal number.

Answer:

Branch target = PC + Offset

Here, PC = 0x00001020 = (4128)₁₀

So, Branch target = (4128)₁₀ + (-4088)₁₀ = (40)₁₀

Question - 12:

address	Instruction	Machine code
0x00001010	Label: add x19, x15, x16	0x010789B3
-		
0x00001046	Beq x10, x11, Label	0x02B50E63
-		
0x00001082	Label: sub x19, x15, x16	0x410789B3

While creating a segment of an assembly code, a mistake happened when the same label name “Label” was used twice.

i) **Figure** out which label the Beq instruction will branch to and show the necessary calculations.

Answer:

0x02B50E63 = 0000 0010 1011 0101 0000 1110 0110 0011

0	000001	01011	01010	000	1110	0	110 0011
imm[12]	imm[10:5]	rs2	rs1	f3	imm[4:1]	imm[11]	Opcode

	imm[12]	imm[11]	imm[10:5]	imm[4:1]
Immediate	0	0	000 001	1110

Immediate = 0000 0001 1110

Here the msb bit of the immediate is 0, meaning it is a positive number.

Since, the immediate is a positive number meaning the jump will be forward.

Therefore, the second label should be chosen.

In case you want to be precise.

$0000\ 0001\ 1110 = (1E)_{16}$
 So the immediate = $+(1E)_{16}$
 Offset = $+(1E)_{16} \times (2)_{16} = +(3C)_{16}$

So, The target jump address = $(00001046)_{16} + (+3C)_{16} = (00001082)_{16}$
 Therefore, the jump will be downward to the address $(00001082)_{16}$

ii) Assuming you figured out the correct “Label”, what will be the machine code to create a branch to the other instruction.

Answer: The other jump is a upward jump. There fore the offset will be negative.

Current Address = $0x00001046$

Target Address = $0x00001010$

Therefore, Offset = $(0x00001010 - 0x00001046) = -0x36$

Forming the immediate,

$0x36 = 0000\ 0011\ 0110$
 $+0x36 = 0\ 0000\ 0011\ 0110$
 $1\ 1111\ 1100\ 1001$
 $ +1$

$-0x36 = 1\ 1111\ 1100\ 1010$

So,

	imm[12]	imm[11]	imm[10:5]	imm[4:1]
Immediate	1	1	111 110	0101

Encoding,

1	111110	01011	01010	000	0101	1	110 0011
imm[12]	imm[10:5]	rs2	rs1	f3	imm[4:1]	imm[11]	Opcode

Machine code = $0xFCB505E3$

Question - 13:

Fill up the following table:

Instruction Set	Machine Code (binary)	Machine Code(Hex)
addi x6, x0, 0	00000000000000000000 001100010011	0x00000313
addi x29, x0, 100	00000110010000000000 111010010011	0x06400E93
LOOP: ld x7, 0(x10)	00000000000001010011 001110000011	0x00053383
add x5, x5, x7	00000000011100101000 001010110011	0x007282B3
addi x10, x10, 8	00000000100001010000 010100010011	0x00850513
addi x6, x6, 1	00000000000100110000 001100010011	0x00130313
blt x6, x29, LOOP	111111111101001101001 00011100011	0xFFD348E3

Use the following table for opcode, f3 and f7 values:

Instruction	Opcode	funct3	funct7
addi	0010011	000	—
ld	0000011	011	—
add	0110011	000	0000000
blt	1100011	100	—

Question - 14:

Consider the below code sequence. Assume X_1 has the value 0x00001232 and X_2 has 1024 (decimal). Calculate the value X_3 stores after executing each line of the code sequence. All the data values are in decimal notation in Table 2.

Answer in Table 1 below and you must show all your calculations in your script.

Code Sequence	Value of X_3
<i>lw X3, 8(X1)</i>	13
<i>slli X3, X3, 2</i>	$13 \times 4 = 52$
<i>Add X3, X3, X2</i>	$52 + 1024 = 1076$

Table - 1

Memory Address	Data (in decimal)
0x00001232	10
0x00001236	12
0x0000123A	13
0x0000123E	14

Table - 2